



Reasoning-Based Traversal of an Unknown Graph: Wikipedia Game AI

Caleb Bailey, James Irwin, Nathan Lim, Kyra Ramirez, Colby Watts

{cjbailey, jirwin06, nlim10, kramir62, cwatts03} @ calpoly.edu

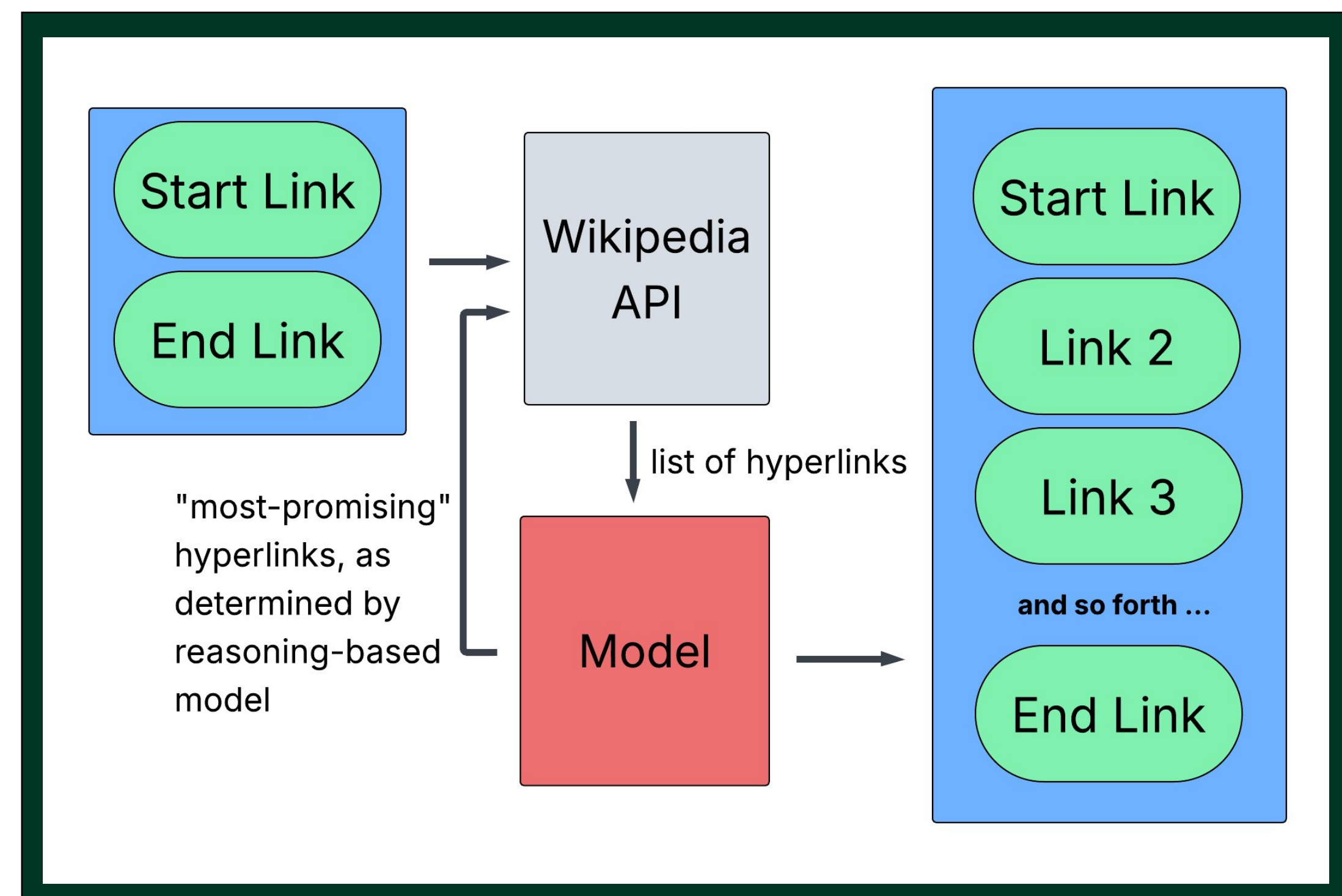


CAL POLY
College of Engineering

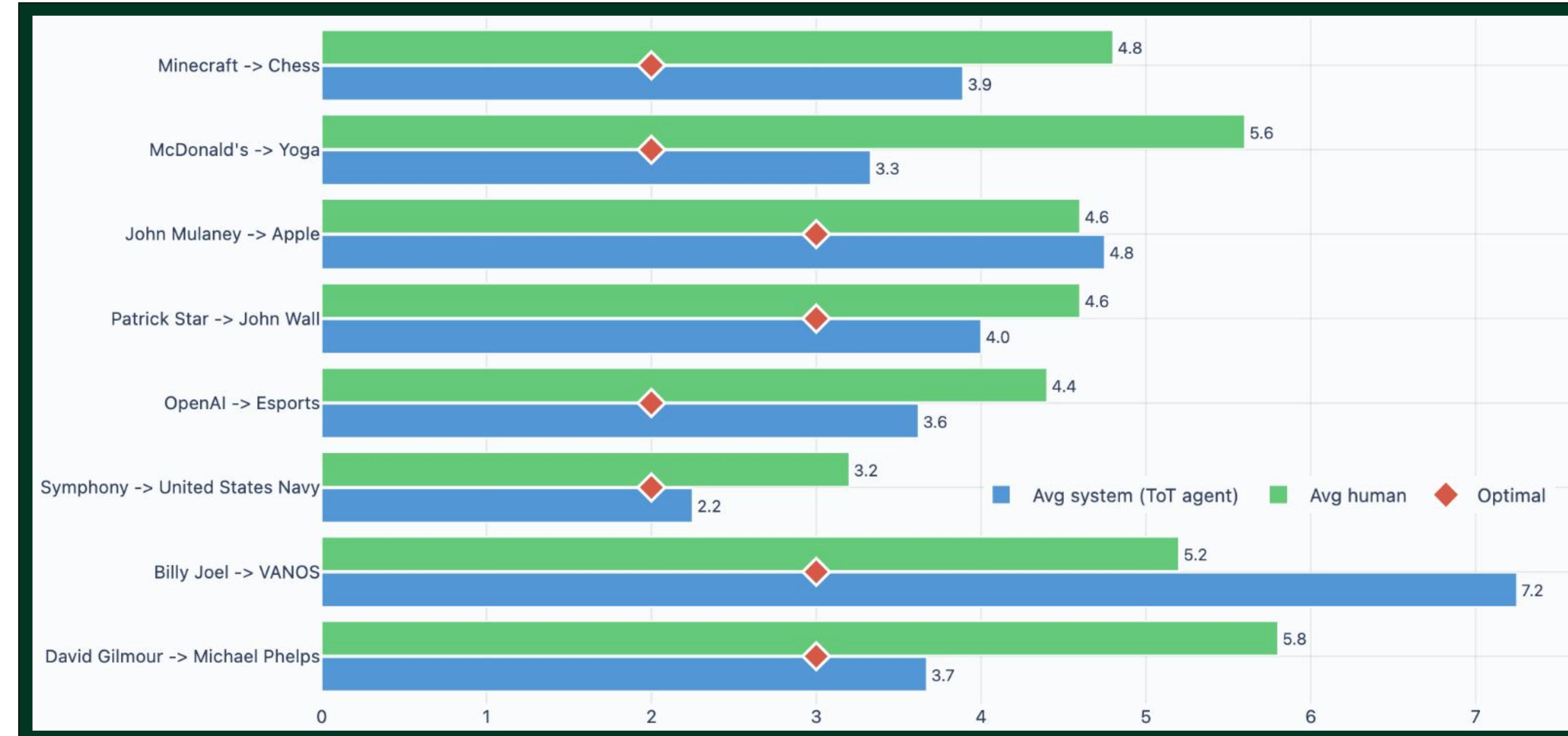
Background

- The Wikipedia Game is a popular internet game where players must navigate between a start and destination Wikipedia page using only the links on the screen—reasoning about which links are likely to lead the user on a path towards the target.
- Wikipedia provides structured, programmatic access to article content and outgoing hyperlinks through MediaWiki Action API, which contains page-level information while showing only the locally available links of the current page.
- This constraint aligns with the formulation of Wikipedia navigation as an **unknown graph search problem**: an LLM agent must incrementally observe and reason about the outgoing links to decide which hyperlink to follow at each step.

System Architecture



- The system operates as an iterative navigation loop between Wikipedia API client and an LLM-based reasoning agent.
- Given a start page and a target page, the system retrieves all outgoing hyperlinks and a small selection of text (first paragraph + subsection headers) from the current page using the MediaWiki API. This information is fed to the currently selected agent, which continually selects links based on which ones are most likely to lead closer to the target page.
- This process repeats until the target page is reached, resulting in a navigation path including the sequence of hyperlinks the agent decided to follow.



Average performance comparison between our system (using our ToT agent) and humans. Y-axis represents start and target page pairs. X-axis represents number of hops.

Project Overview

This project explores the reasoning and planning capabilities of LLM-based agents by implementing a system that navigates Wikipedia purely through hyperlinks. The goal is to evaluate how effectively LLM agents can plan multi-step strategies without access to any explicit graph structure.

Agentic Designs

- Our Default Agent** retrieves the first k number of links from each Wikipedia page, uses basic heuristic scoring with word matching to rank them, and provides these top-k links to the LLM to pick the link that is most likely to lead to the target page.
- Our Planning Agent**, rather than receiving the first k number of links on a page, receives just the Wikipedia page subsection titles and uses an LLM to 1) generate hypotheses about what promising links for an efficient path may be present and 2) generate search queries to find links on the page that correspond to those given hypotheses. We use a hybrid semantic + lexical search with a cross-encoder reranking step to list the resulting links + links closely matching the target page, which the LLM then picks from.

- Our Tree-of-Thoughts (ToT) Agent** uses a hierarchical best-first-search across a tree consisting of promising links within Wikipedia pages that it has explored. Each page is a node, and at every step, it scores the most promising links on a page, and prioritizes exploration of the highest scored pages via a priority queue until the target is reached.

Evaluations and Metrics

- Our evaluation focused on human versus system performance for 8 randomly selected paths (see the exact start and target pages listed in the visual above). A summary of our team's performance versus agent performance is shown in the table below. Optimal paths are the true absolute shortest path between two pages, as calculated using third-party graph-based code.

Agent	Avg Hops	Avg Time (s)	Efficiency (Avg Hops / Avg Optimal)	Paths Ever Optimal
Human	4.78	159.4	0.59	2/8
Default	4.94	58.4	0.61	2/8
Planning	4.58	178.0	0.65	3/8
ToT	3.94	44.1	0.67	4/8

User Interface

Wikipedia Game AI (Developed by The Code Ninjas)

Start Wikipedia page title: Target Wikipedia page title:

Agent: LLM candidates: Model:

Result
Success in 3 hops (14.55s) Target: Chess
Each hop prints the exact anchor text you can Ctrl+F on the current Wikipedia page, then click.
Current path:
Minecraft --["Genres"]--> Video game genre --["turn based"]--> Turn-based game --["chess"]--> Chess
Steps (links clicked):
Start: Minecraft
Step 1: Genres
Step 2: turn based
Step 3: chess
Destination: Chess

Path Statistics
Optimal shortest path (via Wikidata) 2 hops
Minecraft --> Scrabble --> Chess
Agent Final Path Performance Metrics

Metric	Value
Optimal path length	2
LLM path length	3
Efficiency (optimal / LLM)	66.7%
Number of shortest paths that exist	5
Time elapsed	14.55s

Future Work

Over the course of this project, our agent significantly improved its reasoning and navigation performance. Future work could focus on prompt engineering for the agents, modifying agent hyperparameters, better conveying the agent's thought process to the user, and more clearly visualizing the path the agent takes through Wikipedia. This would be accomplished by:

- Running evaluations with modified prompts using researched prompt engineering techniques
- Performing a grid search over possible agent hyperparameters (candidate links, ToT branching, etc.)
- Visualizing the agent's navigation path directly on Wikipedia pages by highlighting visited links and showing the travel sequence, as an output video file.
- Adding a live graph view of explored pages and candidate links during navigation.

References

- [1] Wikipedia/Mediawiki API documentation https://www.mediawiki.org/wiki/API:Action_API
- [2] Search-on-Graph: Iterative Informed Navigation for Large Language Model Reasoning on Knowledge Graphs <https://arxiv.org/abs/2510.08825>
- [3] Evaluating Large Language Models on Wikipedia Graph Navigation: Insights from the WikiGame <https://aclanthology.org/2025.clicit-1.63.pdf>
- [4] LangGraph Documentation <https://docs.langchain.com/oss/python/langgraph/overview>